



2. 회원가입·로그인 (이메일·소셜 OAuth)

담당: 이정하

개요

이메일 기반 회원가입/로그인/로그아웃과 Google·GitHub 소셜 로그인(OAuth)까지, 계정이 만들어지고 로그인되는 모든 경로를 다룬다. 로그인 실패 사유 비노출(계정 열거 차단), 강력한 비밀번호 정책, 4테이블 원자적 회원가입이 핵심이며, 소셜 가입도 같은 원칙을 따른다.

- 구현 파일: `auth/service/AuthService.java`, `auth/service/SessionService.java`, `auth/controller/AuthController.java`, `auth/form/SignupForm.java`, `common/validation/StrongPassword.java`, `PasswordPolicy.java`, `auth/service/OAuthService.java`, `config/OAuthProperties.java`

핵심 포인트

- 이메일 미존재와 비밀번호 불일치를 모두 `AUTH_INVALID_CREDENTIALS` 하나로 응답 — 가입 이메일 존재 여부를 알아낼 수 없음
- 비밀번호 검증 전에 계정 상태를 먼저 확인해 정지 계정은 `AUTH_SUSPENDED` 로 분리
- 로그아웃은 `removeAttribute` 가 아니라 `session.invalidate()` 로 세션 ID 자체를 폐기 (**session fixation 방지**)
- 로그인 후 redirect 파라미터는 `/` 로 시작하고 `//` 가 아닌 내부 경로만 허용 (**open redirect 방지**)
- `@StrongPassword` (8~16자, 대·소문자+숫자+특수문자) + `@AssertTrue` 커스텀 검증(비밀번호에 닉네임·이메일 아이디 포함 금지, 비밀번호 확인 일치)을 폼 클래스에 선언
- 회원가입은 `users` (계정) → `user_credentials` (BCrypt 해시) → `user_settings` (테마·알림 기본값) → `user_learning_profile` (관심 과목, BRONZE 시작) 4개 row를 하나의 `@Transactional` 로 생성

주요 코드

위치: `auth/service/AuthService.java` 60~72행 (`login`)

```
private UserSessionResponse login(HttpSession session, String email, String rawPassword) {
    LoginUserRow row = userCredentialMapper.findByLoginEmail(email)
        .orElseThrow(() -> new BusinessException(ErrorCode.AUTH_INVALID_CREDENTIALS));
    if (!STATUS_ACTIVE.equals(row.getStatus())) {
        throw new BusinessException(ErrorCode.AUTH_SUSPENDED);
    }
    if (!passwordEncoder.matches(rawPassword, row.getPasswordHash())) {
        throw new BusinessException(ErrorCode.AUTH_INVALID_CREDENTIALS);
    }

    SessionUser sessionUser = toSessionUser(row);
    sessionService.saveUser(session, sessionUser);
    return sessionService.toLoginResponse(sessionUser);
}
```

위치: `auth/service/SessionService.java` 34~37행 (`logout`)

```
public void logout(HttpSession session) {
    // removeAttribute만 하면 세션 ID가 유지돼 세션 고정(session fixation) 공격에 취약 → 세션 자체를 폐기
    session.invalidate();
}
```

위치: `auth/form/SignupForm.java` 14~31행 (검증 선언부)

```
public class SignupForm {
    @NotBlank @Email private String email;
```

```

        @NotBlank @StrongPassword private String password;    //
        8~16자, 대소문자+숫자+특수문자
        @NotBlank private String confirmPassword;
        @NotBlank @Size(min = 2, max = 50) private String nickname;

        private Long primarySubjectId;
        private String learningGoal;

        @AssertTrue(message = "비밀번호에 닉네임이나 이메일 아이디를
        포함할 수 없습니다.")
        public boolean isPasswordNotContainingProfile() {
            return !PasswordPolicy.containsProfileInfo(password, nickname, email);
        }

        @AssertTrue(message = "비밀번호 확인이 일치하지 않습니다.")
        public boolean isPasswordConfirmed() {
            if (password == null || confirmPassword == null) return true;
            return password.equals(confirmPassword);
        }
    }
}

```

위치: `auth/service/AuthService.java` 115~155행 (`signup`, setter 나열 일부 축약)

```

private UserSessionResponse signup(HttpSession session, String email, String rawPassword,
    String nickname, Long primarySubjectId, String learningGoal) {
    if (userCredentialMapper.findByLoginEmail(email).isPresent()) {
        throw new BusinessException(ErrorCode.AUTH_EMAIL_DUPLICATED);
    }
    if (userMapper.existByNickname(nickname)) {
        throw new BusinessException(ErrorCode.AUTH_NICKNAME_DUPLICATED);
    }
}

```

```

        User user = new User(); // 1) use
rs: 계정 본체
        user.setRole(SessionUser.ROLE_USER);
        user.setStatus(STATUS_ACTIVE);
        userMapper.insert(user);

        UserCredential credential = new UserCredential(); //
2) 비밀번호는 BCrypt 해시로만 저장
        credential.setUserId(user.getUserId());
        credential.setPasswordHash(passwordEncoder.encode(rawPa
ssword));
        userCredentialMapper.insert(credential);

        UserSetting setting = new UserSetting(); // 3) use
r_settings: 테마·알림 기본값
        setting.setTheme("SYSTEM");
        userSettingMapper.insert(setting);

        UserLearningProfile profile = new UserLearningProfile
(); // 4) 학습 프로필: BRONZE 시작
        profile.setPrimarySubjectId(primarySubjectId);
        profile.setCurrentLevelCode("BRONZE");
        userLearningProfileMapper.insert(profile);
        // @Transactional – 넷 중 하나라도 실패하면 전부 롤백
        return sessionService.toSignupResponse(toSessionUser(us
er));
    }

```

소셜 로그인 (OAuth: Google·GitHub)

이메일 가입 외에 Google/GitHub 계정으로도 가입·로그인할 수 있다. 진입 경로는 달라도 위의 이메일 가입과 같은 원칙을 따른다 — 가입이 중간에 끊겨도 반쪽짜리 계정을 남기지 않고, 로그인 전에 계정 상태를 먼저 확인한다.

- 소셜 로그인 시작 시마다 서버가 난수(`state`)를 세션에 저장하고, provider 콜백 시 같은 값인지 대조 — 값이 다르면 위조 요청으로 간주해 즉시 중단 (CSRF성 위조 차단)
- 이미 연동된 소셜 계정이면 그 자리에서 바로 로그인

- 처음 온 소셜 계정은 **바로 회원을 만들지 않는다** — provider에서 받은 정보(이메일·이름)를 세션에만 잠시 보관하고, 닉네임·관심 과목 입력까지 마쳐야 위에서 본 4테이블 트랜잭션으로 회원이 생성된다 (가입 중 이탈해도 반쪽짜리 계정이 남지 않음)
- 탈퇴했던 계정이 같은 소셜 계정으로 재가입할 때도 옛 연동 기록을 새 회원에 다시 연결해 재가입을 자연스럽게 허용
- 인증 정책: 이메일/Google/GitHub는 서로 독립 계정이며 동일 이메일이어도 자동 병합하지 않음

위치: `auth/service/OAuthService.java` 82~118행 (`handleLoginCallback`, 탈퇴 분기·이메일 대체값 생성부 축약)

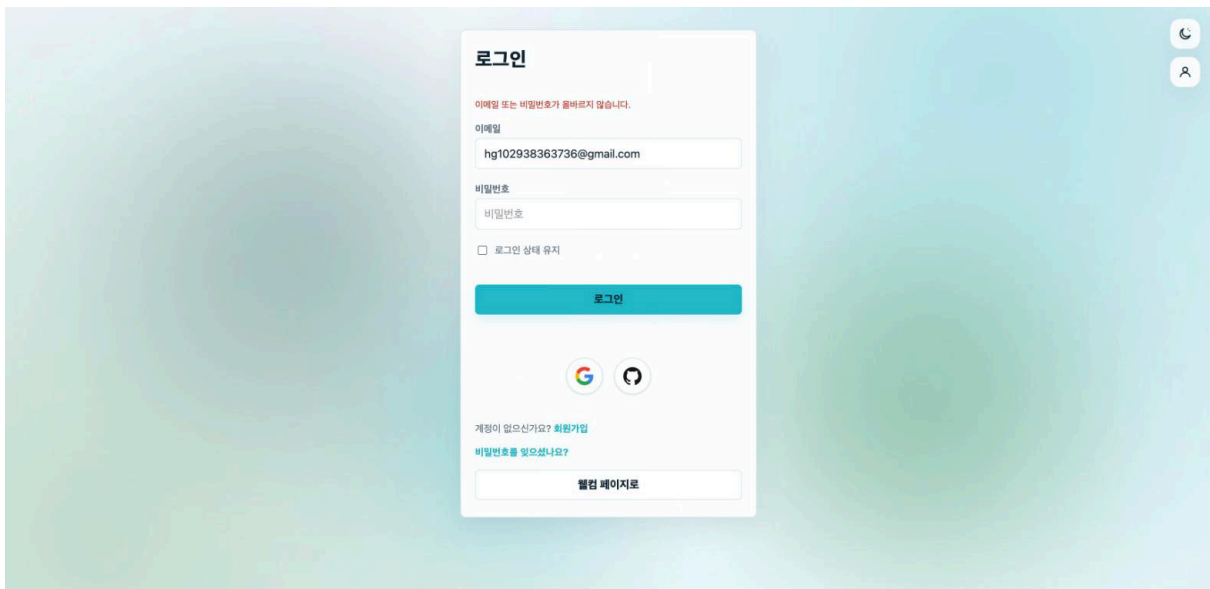
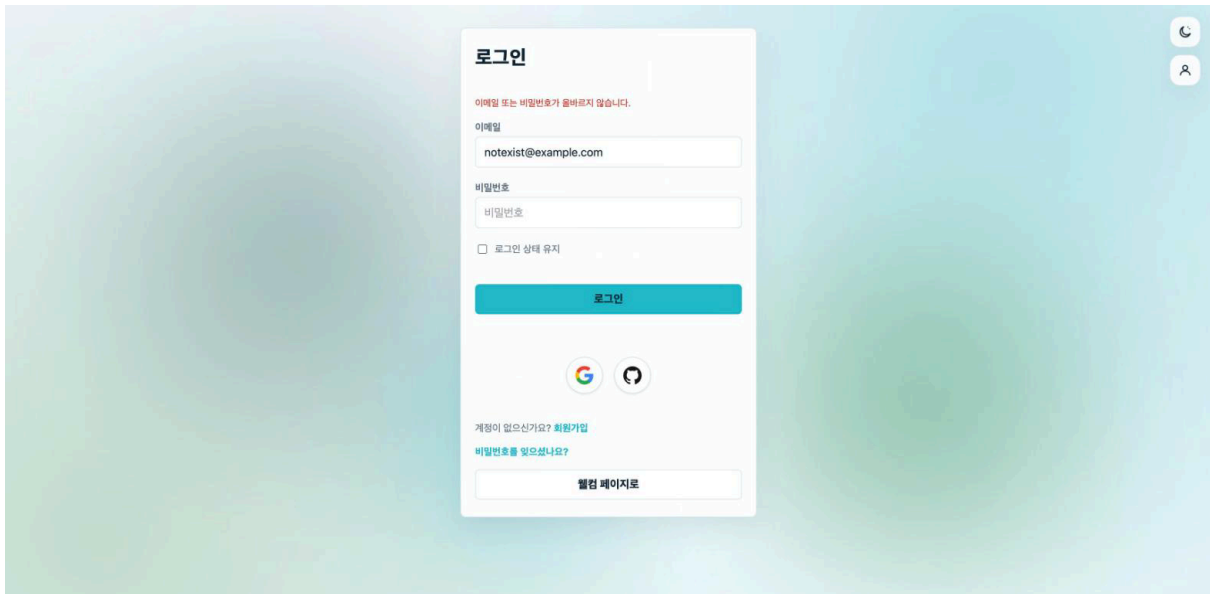
```
// callback: state 검증 → 연동 계정이면 로그인, 아니면 가입 대기
// 상태로
validateState(session, provider, state);
session.removeAttribute(OAUTH_STATE_KEY);

OAuthUserInfo info = fetchUserInfo(provider, cfg, code);
SocialAccount linked = socialAccountMapper
    .findByProviderAndProviderUserId(provider, info.providerUserId())
    .orElse(null);
if (linked != null && !STATUS_WITHDRAWN.equals(owner.getStatus())) {
    sessionService.saveUser(session, toSessionUser(owner));
    return sessionUser.defaultRedirectPath();
}

// 신규·재가입 → DB 생성 없이 세션에만 임시 보관 후 회원가입 화면으로
PendingSocialSignup pending = new PendingSocialSignup(
    provider, info.providerUserId(), resolvedEmail, info.name(), info.emailVerified());
session.setAttribute(PENDING_SOCIAL_SIGNUP_KEY, pending);
return "/oauth/signup";
```

시연 화면

1. `/login` — 미가입 이메일로 로그인 시도 → "이메일 또는 비밀번호가 올바르지 않습니다."
2. `/login` — 가입된 이메일 + 틀린 비밀번호 → **동일한 에러 문구** (계정 열거 차단 확인)
3. `/signup` — 회원가입 폼
4. `/signup` — 비밀번호 규칙 위반 + 비밀번호 확인 불일치 검증 에러
5. `/login` 하단의 Google/GitHub 로고 버튼 → 신규 소셜 계정이면 `/oauth/signup` 가입 완성 화면(닉네임·관심 과목 입력)으로 이어짐



회원가입

이메일 가입

이메일

newuser@example.com

비밀번호

Knowva1234!

비밀번호 확인

비밀번호 재입력

닉네임

신규유저

관심 과목 (선택)

선택 안 함

학습 목표 (선택)

Java 백엔드 학습

가입하기

이미 계정이 있으신가요? 로그인

회원가입

이메일 가입

이메일

docs.test@example.com

비밀번호

Knowva1234!

비밀번호는 8~16자이며 영문 대소문자, 숫자, 특수문자를 모두 포함해야 합니다.

비밀번호 확인

비밀번호 재입력

비밀번호 확인이 일치하지 않습니다.

닉네임

문서테스트

관심 과목 (선택)

선택 안 함

학습 목표 (선택)

Java 백엔드 학습

가입하기



아이디가 없으신가요? [회원가입](#)